GUÍA DE CONFIGURACIÓN

Entorno de Desarrollo con

CLAUDE CODE

Versión 1.0 • Febrero 2026

CONTENIDO

01 Introducción y Objetivos
02 Instalación y Configuración Inicial
03 El Archivo CLAUDE.md
04 Sesiones Paralelas con Git Worktrees
05 El Modo Plan
06 Subagentes y Delegación
07 MCP Servers
08 Skills y Comandos Personalizados
09 Hooks y Automatización
10 Revisión de Código
11 Configuración del Terminal
12 Mejores Prácticas Avanzadas
13 Integración con CI/CD
14 Referencia Rápida

01 INTRODUCCIÓN Y OBJETIVOS

Esta guía está diseñada para equipos de desarrollo de software que desean integrar Claude Code en su flujo de trabajo diario. Claude Code no es simplemente un autocompletador de código; es un agente autónomo que vive en tu terminal, comprende tu base de código completa y puede ejecutar tareas mientras tú supervisas.

¿Por qué Claude Code?

- ⚡ Integración directa en el terminal sin servidores adicionales
- 🧠 Comprensión profunda del contexto y dependencias del proyecto
- 🤖 Autonomía controlada: ejecuta comandos, gestiona git, crea PRs
- 💭 Extended Thinking para decisiones arquitectónicas complejas

Objetivos de esta Guía

- ☑ Establecer un entorno de desarrollo profesional y reproducible
- ☑ Maximizar productividad con sesiones paralelas y worktrees
- ☑ Implementar patrones probados por equipos de alto rendimiento
- ☑ Integrar Claude Code en el ciclo de vida de desarrollo

> 🚀 **TIP**
>
> Ejecutar 3-5 sesiones de Claude en paralelo usando git worktrees es, según el equipo de Claude Code, el desbloqueador de productividad más significativo. Esta guía te enseñará cómo implementarlo.

02 INSTALACIÓN Y CONFIGURACIÓN INICIAL

Requisitos del Sistema

Componente	Requisito	Notas
Sistema Operativo	macOS, Linux, Windows (WSL)	WSL 2 recomendado para Windows
Git	Versión 2.x+	Configurado con usuario y email
Terminal	Moderna (Ghostty, iTerm2)	Con soporte Unicode
Cuenta	Claude Pro, Max o API	OAuth o API Key

Instalación Nativa (Recomendada)

Anthropic ofrece instaladores nativos que no requieren Node.js:

macOS / Linux

```
Terminal
curl -fsSL https://claude.ai/install.sh | bash
```

Windows (CMD)

Claude ofrece una opción para instalarlo desde la CMD, pero hay que realizar una serie de pasos para sortear las limitaciones corporativas.

En primer lugar, nos descargamos el siguiente archivo a mano accediendo a la siguiente dirección en el navegador:

https://claude.ai/install.cmd

Abrimos una CMD y desactivamos la validación de certificados Sser SL para conexiones HTTPS y TLS. Después, desde la ubicación del archivo que acabamos de descargar, lo ejecutamos.

```
CMD
set NODE_TLS_REJECT_UNAUTHORIZED=0
bootstrap.cmd
```

Tras la instalación, debemos crear una variable de entorno para indicarle a Claude dónde se encuentra el path de Git y añadir la ruta del ejecutable de Claude al path de Windows. Ahora vamos a utilizar la PowerShell. Buscamos la ruta de la instalación de Git que, dependiendo de si se ha instalado con Scoop o de la forma habitual, lo encontraremos en la carpeta de usuario o en archivos de programa.
Este paso se realiza desde una PowerShell porque la opción de editar las variables de entorno y el path desde el panel de control de Windows está capada.

```
PowerShell

[Environment]::SetEnvironmentVariable("CLAUDE_CODE_GIT_BASH_PATH",
"C:\Users\mi.usuario\scoop\apps\git\current\bin\bash.exe", "User")
```

```
PowerShell

$currentPath = [Environment]::GetEnvironmentVariable("Path", "User")
$newPath = $currentPath + ";C:\Users\mi.usuario\.local\bin"
[Environment]::SetEnvironmentVariable("Path", $newPath, "User")
```

Si nos encontramos con un problema de certificado al utilizar Claude, ponemos la variable de entorno NODE_EXTRA_CA_CERTS. Al utilizar setx, se mantendrá la variable de entorno de forma persistente en vez de quedarse solo para la sesión actual de la consola.

```
CMD

setx NODE_EXTRA_CA_CERTS C:\ProgramData\Arup\Certificates\curl-ca-bundle.crt
```

Windows vía WSL (alternativa)

Si ya usas WSL (Windows Subsystem for Linux), puedes instalar Claude Code directamente dentro de tu distribución Linux. Esto es especialmente útil si tu flujo de desarrollo ya vive en WSL:

```
Desde tu terminal WSL (Ubuntu, Debian, etc.)

curl -fsSL https://claude.ai/install.sh | bash
```

Nota: Al instalarlo en WSL, Claude Code tendrá acceso nativo al sistema de archivos Linux (/home/tu-usuario/) y podrá interactuar con tus proyectos dentro de WSL sin fricciones. Los archivos en el lado Windows son accesibles desde /mnt/c/, aunque con menor rendimiento de I/O.

Verificación

```
Terminal
```

```
claude --version
claude doctor    # Diagnostica problemas comunes
```

> 💡 **TIP**
> Ejecuta 'claude doctor' después de cada instalación. Este comando auto-detecta la mayoría de problemas de configuración y sugiere soluciones específicas.

Métodos de Autenticación

1	→	2	→	3	→	4
Instalar		Autenticar		Configurar		¡Listo!

- **Claude Pro/Max:** Autenticación vía OAuth con tu cuenta de claude.ai
- **API Key:** Para uso programático o en CI/CD
- **Teams/Enterprise:** Gestión centralizada de facturación

Autenticación

```
# Primera vez: Se abre navegador para OAuth
cd tu-proyecto
claude

# O con API Key
export ANTHROPIC_API_KEY='sk-ant-...'
claude
```

03 EL ARCHIVO CLAUDE.MD

CLAUDE.md es el archivo de configuración más importante para Claude Code. Se lee automáticamente al inicio de cada conversación, proporcionando contexto persistente sobre tu proyecto.

> ⚠️ **IMPORTANTE**
> El nombre del archivo es sensible a mayúsculas/minúsculas. Debe ser exactamente CLAUDE.md (CLAUDE en mayúsculas, .md en minúsculas).

Generación Inicial

```
• • • Terminal

# En el directorio raíz de tu proyecto
claude
/init    # Genera CLAUDE.md basado en tu proyecto
```

El comando /init analiza tu codebase, detecta sistemas de build, frameworks de testing y patrones de código. Úsalo como punto de partida.

Ubicaciones del Archivo

- 🏠 ~/.claude/CLAUDE.md → Configuración global (todos los proyectos)
- 📁 ./CLAUDE.md → Configuración del proyecto (raíz del repo)
- 📂 ./subdir/CLAUDE.md → Configuración específica por módulo

Configuración Avanzada

Las siguientes directivas transforman a Claude de un asistente básico en un agente de desarrollo autónomo y disciplinado. Copia estas secciones en tu CLAUDE.md:

1. Plan Mode por Defecto

CLAUDE.md

```
## Plan Mode Default
- Enter plan mode for ANY non-trivial task (3+ steps or
  architectural decisions)
- If something goes sideways, STOP and re-plan immediately
  — don't keep pushing
- Use plan mode for verification steps, not just building
- Write detailed specs upfront to reduce ambiguity
```

> 💡 **POR QUÉ FUNCIONA**
>
> Forzar planificación antes de ejecución reduce errores en un 60-70%. Si algo sale mal, re-planificar inmediatamente evita acumular deuda técnica.

2. Estrategia de Subagentes

CLAUDE.md

```
## Subagent Strategy
- Use subagents liberally to keep main context window clean
- Offload research, exploration, and parallel analysis
- For complex problems, throw more compute via subagents
- One task per subagent for focused execution
```

3. Bucle de Auto-Mejora

CLAUDE.md

```
## Self-Improvement Loop
- After ANY correction from user, update tasks/lessons.md
- Write rules for yourself that prevent the same mistake
- Ruthlessly iterate until mistake rate drops
- Review lessons at session start for relevant project
```

> 🚀 **PRODUCTIVIDAD COMPUESTA**

> Este patrón crea un ciclo virtuoso: cada corrección se convierte en una regla permanente. Con el tiempo, la tasa de errores cae dramáticamente.

4. Verificación Antes de Completar

CLAUDE.md

```
## Verification Before Done
- Never mark a task complete without proving it works
- Diff behavior between main and your changes when relevant
- Ask yourself: "Would a staff engineer approve this?"
- Run tests, check logs, demonstrate correctness
```

5. Exigir Elegancia (Balanceado)

CLAUDE.md

```
## Demand Elegance (Balanced)
- For non-trivial changes: pause and ask,
  "Is there a more elegant way?"
- If a fix feels hacky: "Knowing everything I know now,
  implement the elegant solution."
- Skip this for simple, obvious fixes — don't over-engineer
- Challenge your own work before presenting it
```

6. Corrección Autónoma de Bugs

```
CLAUDE.md

## Autonomous Bug Fixing
- When given a bug report: just fix it. No hand-holding
- Point at logs, errors, failing tests — then resolve them
- Zero context switching required from the user
- Go fix failing CI tests without being told how
```

> **OBJETIVO**
> El desarrollador puede pegar un bug report y decir "fix". Claude debe diagnosticar, localizar, corregir y verificar sin preguntas intermedias.

7. Gestión de Tareas

```
CLAUDE.md

## Task Management
- Plan First: Write plan to tasks/todo.md with checkable items
- Verify Plan: Check in before starting implementation
- Track Progress: Mark items complete as you go
- Explain Changes: High-level summary at each step
- Document Results: Add review section to tasks/todo.md
- Capture Lessons: Update tasks/lessons.md after corrections
```

8. Principios Core

```
CLAUDE.md

## Core Principles
- Simplicity First: Make every change as simple as possible.
  Impact minimal code.
- No Laziness: Find root causes. No temporary fixes.
  Senior developer standards.
- Minimal Impact: Changes should only touch what's necessary.
  Avoid introducing bugs.
```

Estructura de Carpetas Recomendada

```
Estructura

proyecto/
```

```
├── CLAUDE.md           # Configuración principal
├── tasks/
│   ├── todo.md         # Plan y checklist actual
│   └── lessons.md      # Lecciones aprendidas
├── docs/
│   └── architecture.md # Referencia de arquitectura
└── .claude/
    ├── commands/       # Slash commands personalizados
    └── agents/         # Subagentes del proyecto
```

Mejores Prácticas para CLAUDE.md

Práctica	Descripción
Mantener conciso	Cada línea compite por atención. Solo lo universalmente aplicable.
Actualizar siempre	Después de cada corrección: "Actualiza CLAUDE.md para evitar esto"
Revelación progresiva	Indica dónde encontrar docs detallados, no los incluyas todos
Usar linters	CLAUDE.md no reemplaza herramientas automáticas
Versionado	Commitea CLAUDE.md al repo para que todo el equipo itere

04 SESIONES PARALELAS CON GIT WORKTREES

🚀 **MAYOR MULTIPLICADOR DE PRODUCTIVIDAD**

Ejecutar 3-5 sesiones de Claude en paralelo usando git worktrees es el mayor desbloqueador de productividad según el equipo de Claude Code. Algunos configuran aliases (za, zb, zc) para saltar entre sesiones en una sola tecla.

¿Qué son los Git Worktrees?

Git worktrees permiten tener múltiples directorios de trabajo del mismo repositorio, cada uno en un branch diferente. Cada worktree tiene su propio estado de archivos independiente.

1	→	2	→	3	→	4
Repo Principal		Worktree A		Worktree B		Worktree C

Comandos Esenciales

Git Worktrees

```
# Crear worktrees para desarrollo paralelo
git worktree add ../proyecto-feature-login -b feature/login main
git worktree add ../proyecto-feature-dashboard -b feature/dashboard main
git worktree add ../proyecto-hotfix -b hotfix/critical main

# Listar worktrees activos
git worktree list

# Eliminar worktree cuando termines
git worktree remove ../proyecto-feature-login
```

Flujo de Trabajo Paralelo de ejemplo

Terminal	Worktree	Tarea	Estado Claude
Terminal 1	Principal	Desarrollo activo	Interactivo
Terminal 2	feature-login	Feature nueva	Autónomo
Terminal 3	hotfix	Bug crítico	Supervisado

Aliases de ejemplo

```
~/.bashrc

# Añadir a ~/.bashrc o ~/.zshrc

# Navegación rápida entre worktrees
alias za='cd ~/proyectos/app-worktree-a && claude'
alias zb='cd ~/proyectos/app-worktree-b && claude'
alias zc='cd ~/proyectos/app-worktree-c && claude'

# Función para crear worktree + iniciar Claude
wt() {
  git worktree add ../$(basename $(pwd))-$1 -b feature/$1 main
  cd ../$(basename $(pwd))-$1
  claude
}
```

⚠️ **CONSIDERACIONES**

Cada worktree necesita su propio setup de entorno (npm install, venv, etc.). Gestiona puertos y bases de datos para evitar conflictos entre sesiones paralelas.

05 EL MODO PLAN: ANTES DE ESCRIBIR CÓDIGO

Una de las prácticas más importantes es invertir tiempo en planificación antes de que Claude escriba código. El equipo de Claude Code utiliza extensivamente este patrón.

Activar el Modo Plan

Atajos de Modo Plan

→ `Shift+Tab (x2)` Activa/desactiva modo Plan
→ `claude --plan` Inicia directamente en modo Plan
→ `Escape` Interrumpe y permite dar feedback

En modo Plan, Claude analiza, investiga y planifica pero no puede modificar archivos. Es como ponerlo en "modo arquitecto".

Patrón: Plan → Revisión → Ejecución

1	→	2	→	3	→	4
Planificar		Revisar		Aprobar		Ejecutar

1. Claude escribe el plan detallado
2. Otro Claude (o tú) lo revisa como Staff Engineer
3. Si algo no está claro, vuelve a modo Plan y replanifica
4. Una vez aprobado, Claude ejecuta la implementación

Guardar Planes como Archivos

Prompt de Planificación

```
"Antes de implementar, escribe un plan detallado en plan.md
con los siguientes elementos:
- Arquitectura propuesta
- Archivos a modificar
- Tests a crear
- Riesgos identificados
- Criterios de aceptación"
```

💡 **BENEFICIO CLAVE**

El archivo plan.md sirve como memoria de trabajo que persiste entre sesiones. Permite retomar el trabajo días después sin empezar de cero.

06 SUBAGENTES Y DELEGACIÓN DE TAREAS

Los subagentes son asistentes especializados con contextos separados. Cada uno tiene su propia ventana de contexto, evitando contaminación y mejorando la precisión.

Cuándo Usar Subagentes

- 🔍 Investigación profunda que consumiría mucho contexto
- ⚡ Tareas paralelas que pueden ejecutarse independientemente
- 🎯 Trabajo especializado (testing, docs, code review)
- 🧹 Mantener el contexto principal limpio para implementación

Invocar Subagentes

Prompts con Subagentes

```
# Añade "use subagents" a tu prompt
"Usa subagentes para investigar cómo nuestro sistema de
autenticación maneja el refresh de tokens."

# Para múltiples tareas paralelas
"Usa múltiples subagentes para revisar estos 5 archivos
en paralelo y reporta los issues encontrados."
```

Crear Subagentes Personalizados

code-reviewer.md

```
# Ubicación: .claude/agents/code-reviewer.md
---
name: code-reviewer
description: Revisa código y sugiere mejoras
model: sonnet  # Opciones: sonnet, opus, haiku, inherit
tools:
  - Read
  - Bash
---

Eres un revisor de código experto. Tu trabajo es:
1. Analizar código en busca de bugs potenciales
2. Identificar problemas de rendimiento
```

```
3. Sugerir mejoras de legibilidad
```

⚡ Comandos de Subagentes

→ `/agents` Abre el gestor de agentes
→ `/agents create` Crea un nuevo subagente
→ `@nombre-agente` Invoca un agente específico

07 MCP SERVERS: HERRAMIENTAS EXTERNAS

El Model Context Protocol (MCP) conecta Claude Code con herramientas y fuentes de datos externas. Es como un adaptador universal para GitHub, bases de datos, APIs y otros sistemas.

Añadir Servidores MCP

```
Terminal

# Añadir servidor MCP a nivel usuario
claude mcp add <nombre> <comando>

# Ejemplos:
claude mcp add playwright npx @playwright/mcp@latest
claude mcp add context7 npx context7-mcp
claude mcp add brave-search env BRAVE_API_KEY=xxx npx @modelcontextprotocol/server-
brave-search
```

MCP Servers de ejemplo

Servidor	Uso Principal	Beneficio
Context7	Documentación de librerías	APIs siempre actualizadas
Playwright	Testing E2E	Tests automatizados de UI
GitHub	Issues, PRs, Actions	Workflow completo de Git
BigQuery/SQL	Consultas de datos	Analytics sin escribir SQL

Gestión de MCP

→ `/mcp` Ver servidores MCP activos
→ `/mcp remove` Eliminar un servidor
→ `/context` Ver uso de contexto por servidor

OPTIMIZACIÓN DE CONTEXTO

Cada servidor MCP consume contexto. Monitorea con /context y elimina los servidores que no uses frecuentemente para mantener espacio para lo importante.

08 SKILLS Y COMANDOS PERSONALIZADOS

Slash Commands

Los comandos personalizados son atajos para flujos de trabajo frecuentes. Se guardan como archivos Markdown:

```
techdebt.md

# Ubicación: .claude/commands/techdebt.md
---
description: Encuentra y elimina código duplicado
---

Analiza el codebase y:
1. Identifica código duplicado
2. Sugiere refactorizaciones
3. Propón abstracciones comunes
```

Skills: Comportamientos Automáticos

Los Skills se activan automáticamente basándose en el contexto. A diferencia de los comandos, Claude decide cuándo invocarlos.

```
SKILL.md

# Ubicación: .claude/skills/testing/SKILL.md
---
name: testing
description: Estándares de testing del proyecto
---

Al escribir tests:
- Usar Jest como framework
- Patrón AAA (Arrange-Act-Assert)
- Mocks para dependencias externas
```

Ideas de Comandos Útiles

Comando	Función

`/sync`	Sincroniza Slack, GDrive, Asana y GitHub en un contexto
`/review`	Revisa cambios como Staff Engineer
`/pr`	Crea PR con descripción detallada
`/explain`	Genera presentación HTML del código
`/diagram`	Dibuja diagramas ASCII de arquitectura

09 HOOKS: AUTOMATIZACIÓN DEL CICLO DE VIDA

Los Hooks ejecutan comandos automáticamente en eventos específicos del ciclo de vida de Claude Code.

Tipos de Hooks

Hook	Cuándo se ejecuta	Caso de uso
PreToolUse	Antes de usar herramienta	Validar comandos bash
PostToolUse	Después de usar herramienta	Ejecutar linter tras editar
Stop	Al terminar tarea	Actualizar documentación
SubagentStop	Al terminar subagente	Consolidar resultados

Configuración de Hooks

settings.json

```
// ~/.claude/settings.json
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "npm run lint --fix"
          }
        ]
      }
    ]
  }
}
```

💡 **CASOS DE USO HABITUALES**

• Ejecutar linter después de cada edición • Validar comandos bash antes de ejecutarlos • Ejecutar tests después de cambios en código • Notificar al equipo cuando se completan tareas

10 REVISIÓN DE CÓDIGO

Una práctica muy efectiva es usar Claude como un revisor de código exigente antes de crear PRs.

Prompts de Revisión Efectivos

Prompts de Revisión

```
# Revisión estricta antes de PR
"Examíname sobre estos cambios y no abras un PR
hasta que yo pase tu evaluación."

# Verificación de funcionamiento
"Demuestra que esto funciona. Compara main vs
la rama de feature."

# Después de una solución débil
"Descarta esto e implementa la versión elegante."
```

Patrón de revisión en paralelo

Usa dos sesiones de Claude: una implementa y otra revisa. Este patrón detecta más problemas.

Sesión 1: Implementador	Sesión 2: Revisor
Escribe el plan	Revisa plan como arquitecto
Implementa features	Busca bugs y edge cases
Crea tests	Verifica cobertura y calidad
Prepara PR	Aprueba o solicita cambios

11 CONFIGURACIÓN DEL TERMINAL

Configuración recomendada de ejemplo

Característica	Configuración	Beneficio
Status Line	`/statusline`	Ver contexto y branch siempre
Dictado	fn fn (macOS)	Hablas 3x más rápido que escribes
Tabs	Color por tarea	Identificación visual rápida
Nombres	Cmd+I (iTerm2)	Nombrar por ticket/feature

Gestión de Sesiones

Comandos de Sesión

→ `/resume` Reanudar sesión anterior (selector interactivo)
→ `claude --resume` Selector desde terminal
→ `claude --from-pr 123` Reanudar sesión de un PR específico

12 MEJORES PRÁCTICAS AVANZADAS

Gestión de Contexto

- **Compactación manual:** /compact <instrucciones> para controlar qué se preserva
- **Subagentes para investigación:** Mantienen el contexto principal limpio
- **Una sesión = Una tarea:** Evita mezclar features en la misma conversación

Permisos y Seguridad

Configuración de Seguridad

→ `/permissions` Gestionar permisos de herramientas
→ `/sandbox` Sandboxing a nivel OS
→ `claude doctor` Verificar configuración de seguridad

Aprendizaje Continuo

- Activa estilo "Explicativo" en /config para entender el 'por qué'
- Pide presentaciones HTML para explicar código complejo
- Solicita diagramas ASCII para entender arquitecturas

Analytics con Claude

Prompt de Analytics

```
"Usa el CLI de BigQuery para consultar métricas
de los últimos 30 días y genera un resumen ejecutivo."
```

13 INTEGRACIÓN CON CI/CD

Ejemplo: Claude en GitHub Actions

claude-review.yml

```yaml
# .github/workflows/claude-review.yml
name: Claude Code Review

on:
  pull_request:
    types: [opened, synchronize]

jobs:
  review:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Run Claude Review
        run: claude "Revisa este PR"
        env:
          ANTHROPIC_API_KEY: ${{ secrets.ANTHROPIC_API_KEY }}
```

Patrones de Automatización

- ☑ Claude fija tests fallidos en CI automáticamente
- ☑ Revisión automática de PRs con comentarios
- ☑ Generación de changelogs desde commits
- ☑ Actualización de docs después de releases

Ciclo de Mejora Continua

1	→	2	→	3	→	4
Bugs en CI		Analizar logs		Actualizar CLAUDE.md		Mejor agente

Revisa regularmente los logs de GitHub Actions para identificar errores comunes y actualiza CLAUDE.md en consecuencia.

14 REFERENCIA RÁPIDA

Comandos de Sesión

Comando	Descripción
`/init`	Genera CLAUDE.md inicial
`/resume`	Reanuda sesión anterior
`/compact`	Compacta contexto con instrucciones
`/clear`	Limpia conversación actual
`/help`	Muestra ayuda de comandos

Comandos de Configuración

Comando	Descripción
`/config`	Configuración de Claude Code
`/permissions`	Gestiona permisos
`/mcp`	Gestiona servidores MCP
`/agents`	Gestiona subagentes
`/context`	Muestra uso de contexto

Atajos de Teclado

Atajo	Acción
`Shift+Tab (x2)`	Activa/desactiva modo Plan
`Escape`	Interrumpe generación actual
`fn fn (macOS)`	Activa dictado por voz
`Ctrl+C`	Cancela operación actual